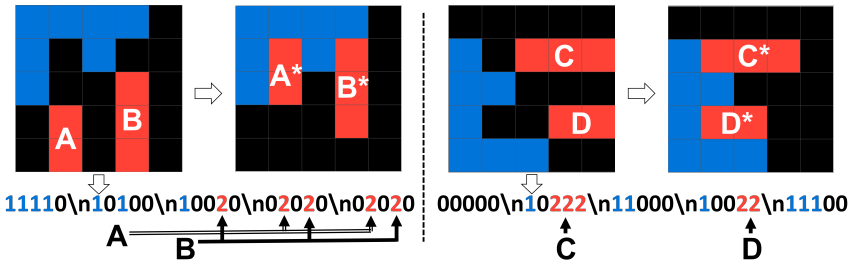


Figure 6: **Object cohesion analysis: two example tasks and their textual representations.** Generated based on the ARC task seen in Figure 2 (Right), the two tasks are identical modulo the 90-degree rotation. Left: objects A and B are vertical and become non-sequential when represented in text. Right: objects C and D are horizontal and become sequential when represented in text.

2.5 Object cohesion

To further understand the limitations of GPT on ARC tasks, we explored the concept of *object cohesion* in text, defined as the *"ability to parse grids into* 'objects' *based on continuity criteria including color continuity or spatial contiguity, and the ability to parse grids into zones, partitions"* (Chollet, 2019). Object cohesion is an integral part of human cognition (Spelke and Kinzler, 2007) and is assumed to be a significant part of the Core Knowledge priors required for ARC solving (Chollet, 2019).

Our objective was to investigate how the textual representation of objects influences GPT's problem-solving capacity. Given that the initial identification and abstraction of objects are pivotal in resolving the ARC (Acquaviva et al., 2022), understanding the impact of textual object depiction on the performance of language models is critical. We discovered that GPT's performance deteriorates significantly when objects are not sequentially represented within the text. To further demonstrate this, we selected tasks with clear horizontal or vertical objects and manipulated them to adopt the opposite orientation. A visualization of the difference between sequential and non-sequential object representation can be found in Figure 6.

For each "horizontal" or "vertical" original ARC task, we generated a rotated version and compared performance in both the horizontal and vertical configurations. An example is visualized in Figure 6 with more examples shown in Appendix C. The results in the leftmost part of Table 2 show a significant performance drop in the vertical case, reinforcing our hypothesis. It is evident that GPT struggles with object cohesion when objects are not sequentially arranged within the text. This insight not only deepens our understanding of the model's limitations but also guides us toward potential solutions. In the following section, we delve further into this finding by generating a new dataset that guarantees object sequentialness, and assessing the performance of LLMs on this dataset.

3 Does reduced task dimensionality improve LLM performance?

We introduce 1D-ARC, a novel variation on the original ARC that reduces its dimensionality to facilitate future research and provide a more approachable benchmark for LLMs. The 1D-ARC maintains the same Core Knowledge priors as the ARC but restricts the dimensionality of the input and output images to one dimension. Consequently, the images comprise only a single row of pixels, significantly reducing the complexity of tasks and enabling all objects to be represented within a single sequence. This modification effectively removes the challenge of maintaining object cohesion in non-sequential text. We visualize some example tasks in Figure 7 but the visualization for the full dataset is included in Appendix A.

The 1D-ARC dataset was strategically designed to adapt transformation types from the original ARC dataset to a one-dimensional format. This methodology effectively preserves the core knowledge priors inherent to the original ARC. An example of the design and generation process for a 1D-ARC task is illustrated in Appendix B.

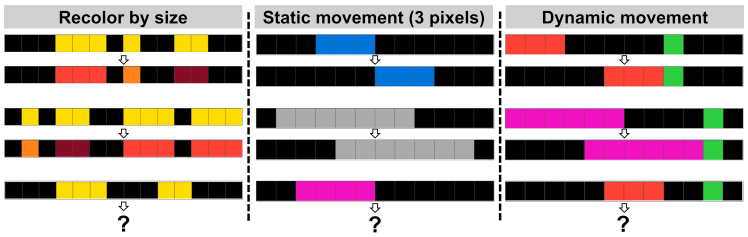


Figure 7: **Example tasks from the 1D-ARC dataset.** Each task is inspired by an ARC task; see Figure 2. From left to right: Recolor by size, Static movement by 3 pixels, Dynamic movement (move the block on the left until it touches the green pixel).

Table 2: **Results for direct-grid approach.** The number of solved tasks is out of 50. The first column is for the 50 tasks from the ARC. The second block of 5 columns is for some 1D-ARC task types; results on the full 1D-ARC can be found in Appendix A. The third block is for three task types (Fill, Move, Pile) with horizontal (H) and vertical (V) variants. All models were provided the same prompts for each task.

LLM	ARC Subset	Move 1 Pixel	Move 3 Pixels	Move Dynamic	Recolor by Size	Denoise	Fill		Move		Pile	
							H	V	H	V	H	V
GPT-3.5	2	10	7	6	2	13	2	0	0	0	2	0
GPT-4	13	33	12	11	14	30	46	1	12	0	32	0
LLAMA2-13B	0	8	3	4	0	4	0	0	0	0	0	0
LLAMA2-70B	2	11	4	5	0	1	0	0	0	0	0	0

Our data generators have been developed to be capable of creating a variety of 1D-ARC tasks. They rely on task-specific parameters such as the maximum width of the 1D sequence, the maximum number of objects, and the maximum size of the objects. This parametric approach ensures tasks can originate from the same foundational concept or transformation yet manifest with varying complexities.

These generators can be seen as "1-concept" in design, each tailored for a specific transformation. However, their modular construction allows for the combination of multiple "1-concept" generators sequentially, leading to multi-concept tasks. As an example, a task could first necessitate the relocation of objects, then require a color transformation of these objects. This layered approach augments the scope and intricacy of the tasks, serving as a comprehensive evaluation of LLM capabilities.

3.1 Results

We used the best-performing prompts from Table 1 for the direct-grid approach and documented the results in the rightmost part of Table 2. Notably, the direct-grid encoding shows a relative improvement in performance on the 1D-ARC as compared to the original ARC. This implies that reducing both task space complexity and the spatial dimensionality of the input-output pairs enhances the LLM's ability to parse and reason with the encoded information.

Even with the significantly simpler 1D-ARC, there is still much room for improvement in performance. While GPT-4 is able to solve some tasks more effectively, it still falls short on others. This finding suggests that providing a sequential representation of objects in text alone may not be sufficient for GPT to effectively solve ARC tasks.

In light of these findings, we next explored the benefits of employing an external tool to perform object abstraction for GPT, thereby completely removing the challenge of object cohesion in text.

4 Enhancing LLM performance with an object-based representation

To address the challenges we have identified thus far and to enhance LLM performance, we propose the integration of an external tool to aid in producing an object representation of a task. More specifically, we leverage the ARGA algorithm (Xu et al., 2023) to execute object abstraction before prompting LLMs for the solution. We note that a task is deemed to be solved correctly when the LLM produces the correct textual representation of the output, given the input produced by the external tool. As such, whenever an LLM is coupled with ARGA, one should think of the integrated "ARGA+LLM" system as tackling the task, rather than the LLM alone.

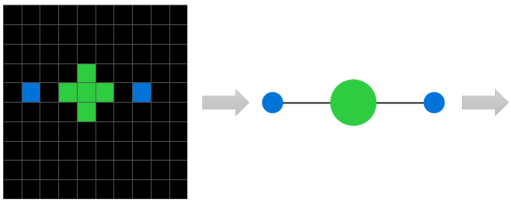
4.1 Object-based textual representation

ARGA is a non-learning approach that aims to solve ARC by first abstracting the images into graph representations and then conducting a search within a Domain-Specific Language (DSL) defining possible changes to the graphs to identify the solution. We leverage the first component of ARGA to acquire a *graph representation* of the images. These graph representations, in which each node (or vertex) corresponds to an object in the image grid and each edge represents relationships between the objects, are subsequently encoded into object-oriented text representations. It is worth noting that ARGA provides a suite of hand-designed abstraction methods to cater to different tasks. In our case, we apply the "best-fit" abstraction, utilizing the abstraction that ARGA deems optimal for generating the solution for each task. Essentially, our objective is to evaluate the effectiveness of an abstraction mechanism that excels at object abstraction. After transforming the images into graphs, we examine two textual representations:

Object Descriptors: This encoding technique presents a list of objects, each corresponding to a node in the graph and its associated attributes; see Figure 8 (Top). It offers a clear and intuitive representation of the image as a set of distinct objects, each carrying its own properties.

Object JSON: On the other hand, the Object JSON encoding method provides a more structured representation of the graph; see Figure 8 (Bottom). This approach involves constructing a JSON list that encapsulates nodes and their corresponding attributes from the graph. The inherent organization of this format simplifies parsing and processing for the LLM, facilitating efficient extraction of pertinent information and relationships between the nodes.

Each encoding approach is further explored with an additional variant that includes edge information from the graph. In ARGA, edge information is utilized to identify relations between objects, as certain transformations applied to objects depend on other objects. For instance, an operation might involve recoloring an object to match the color of its neighbor. In the context of our study, our aim is to investigate whether the inclusion of edge information in the textual representation augments GPT's ability to solve ARC tasks.



```
Image Size:(10, 10)\n
Objects:\n
Object 1: coordinates=[(4, 1)], color= "blue", size=1\n
Object 2: coordinates=[(4, 7)], color= "blue", size=1\n
Object 3: coordinates=[(4, 4), (3, 4), (4, 3), (5, 4), (4, 5)],
          color= "green", size=5
```

```
Image Size:(10, 10)\n
Objects:\n
[{"coordinates": [[4, 1]], "color": "blue", "size": 1},
 {"coordinates": [[4, 7]], "color": "blue", "size": 1},
 {"coordinates": [[3, 4], [4, 3], [4, 4], [4, 5], [5, 4]],
  "color": "green", "size": 5}]
```

Figure 8: **Visualization of object-based textual encodings.** The 2D grid image is first transformed into a graph representation using ARGA. Then, the graph is encoded using the object descriptors representation (Top) or the object JSON representation (Bottom).